

Written Homework Assignment #3

Problems from Ch 7 and 8

You should have read the selections from chapters 7 and 8 of the Stephens text, as specified on the [readings page](#). Having done so will assist you in completing the following assignment.

Note that some of the problems in these assignments are from chapters of the book which you have not been 'officially' assigned to read in detail. However, simply skimming through those chapters will allow you to find the information which will help you answer the problems. This will serve TWO purposes: 1) you will get a bit of research experience as you scan through the chapters; and 2) you'll probably remember a bit more information out of them because you are looking for something specific.

Here are the problems for easy reference.

Problem 7.1, Stephens page 169

The greatest common divisor (GCD) of two integers is the largest integer that evenly divides them both. For example, the GCD of 84 and 36 is 12, because 12 is the largest integer that evenly divides both 84 and 36. You can learn more about the GCD and the Euclidean algorithm, which you can find at en.wikipedia.org/wiki/Euclidean_algorithm. [Hint: It should take you only a few seconds to fix these comments. Don't make a career out of it.]

```
// Use Euclid's algorithm found here
en.wikipedia.org/wiki/Euclidean_algorithm to calculate the GCD.
private long GCD( long a, long b )
{
```

```
for( ; ; )
{
    long remainder = a % b;
    If( remainder == 0 ) return b;
    a = b;
    b = remainder;
};
}
```

Problem 7.2, Stephens page 170

According to your textbook, under what two conditions might you end up with the bad comments shown in the previous code?

The programmer may have started with a detailed description of the algorithm and basically used it as comments.

Or, they may have written comments after writing the code since it is easy to just say what each part is doing without really explaining.

Problem 7.4, Stephens page 170

How could you apply offensive programming to the modified code you wrote for exercise 3? [Yes, I know that problem wasn't assigned, but if you take a look at it you can still do this exercise.]

You can apply offensive programming by checking and validating the inputs and outputs instead of naively assuming they will always be valid.

Problem 7.5, Stephens page 170

Should you add error handling to the modified code you wrote for Exercise 4? Explain your reasoning.

You don't need to add error handling to the GCD method. Since it is a low-level method, errors may be handled differently depending on the context, so you want error handling to come from the calling code.

Problem 7.7, Stephens page 170

Using top-down design, write the highest level of instructions that you would use to tell someone how to drive your car to the nearest supermarket. (Keep it at a very high level.) List any assumptions you make.

Assumptions:

You have a driver's license and are physically able to drive

The car has gas and is working

There is a supermarket in driving distance and you know how to get there

There is parking at the supermarket

Steps:

Enter the car and sit in the driver's seat

Map your destination

Start the car

Drive, follow the instructions of your navigation device and the laws of the road

Park at the supermarket

Problem 8.1, Stephens page 199

Two integers are *relatively prime* [or *coprime*] if they have no common factors other than 1. For example, $21 = 3 \times 7$ and $35 = 5 \times 7$ are *not* relatively prime because they are both divisible by 7. However, integers $8 = 2 \times 4$ and $9 = 3 \times 3$ ARE relatively prime because the only common factor they have is 1 [even though NEITHER of them is prime by itself!] By definition -1 and 1 are relatively prime to every integer, and they are the only numbers relatively prime to 0.

Suppose you've written an efficient `isRelativelyPrime()` method that takes two integers between -1 million and 1 million as parameters and returns `true` if they are relatively prime. Use either your favorite programming language or pseudocode to write a program that tests the `isRelativelyPrime()` method. [Hint: You may find it useful to write the `isRelativelyPrime()` method itself as well.]

```
def run_tests():  
    assert is_relatively_prime(8, 9) == True  
    assert is_relatively_prime(21, 35) == False  
  
    assert is_relatively_prime(1, 100) == True  
    assert is_relatively_prime(-1, 999) == True  
    assert is_relatively_prime(1, -500) == True  
  
    assert is_relatively_prime(0, 1) == True  
    assert is_relatively_prime(1, 0) == True  
    assert is_relatively_prime(0, 2) == False  
    assert is_relatively_prime(0, -1) == True  
    assert is_relatively_prime(0, 0) == False # by definition  
  
    assert is_relatively_prime(-8, 9) == True  
    assert is_relatively_prime(-12, -25) == True  
  
    assert is_relatively_prime(5, 5) == False  
    assert is_relatively_prime(1, 1) == True  
    assert is_relatively_prime(-1, -1) == True  
  
    assert is_relatively_prime(10, 20) == False  
    assert is_relatively_prime(13 * 5, 13 * 7) == False  
  
    assert is_relatively_prime(1_000_000, 999_999) == True  
    assert is_relatively_prime(1_000_000, 500_000) == False
```

```
assert is_relatively_prime(14, 15) == True
assert is_relatively_prime(35, 64) == True
```

Problem 8.3, Stephens page 199

1. What testing techniques did you use for the program in Exercise 8.1? [Exhaustive, black-box, white-box, or gray-box?]
2. Which ones *could* you use and under what circumstances? [Justify your answer with a short paragraph to explain.]

I used black-box testing, since test cases were designed based on inputs and outputs without considering the actual code.

Exhaustive testing was not feasible due to the extremely large input range. However, white-box and gray-box testing could be used if the implementation details were known, and exhaustive testing could only be used if the input range was significantly smaller.

Problem 8.5, Stephens page 199 - 200

the following code shows a C# version of the `areRelativelyPrime()` method and the `GCD` method it calls.

```
// Return true if a and b are relatively prime.
private bool areRelativelyPrime( int a, int b )
{
    // Only 1 and -1 are relatively prime to 0.
    if( a == 0 ) return ((b == 1) || (b == -1));
    if( b == 0 ) return ((a == 1) || (a == -1));

    int gcd = GCD( a, b );
    return ((gcd == 1) || (gcd == -1));
}
```

```
// Use Euclid's algorithm to calculate the
```

```

// greatest common divisor (GCD) of two numbers.
// See https://en.wikipedia.org/wiki/Euclidean_algorithm
private int GCD( int a, int b )
{
    a = Math.abs( a );
    b = Math.abs( b );

    // if a or b is 0, return the other value.
    if( a == 0 ) return b;
    if( b == 0 ) return a;

    for( ; ; )
    {
        int remainder = a % b;
        if( remainder == 0 ) return b;
        a = b;
        b = remainder;
    };
}

```

The `areRelativelyPrime()` method checks whether either value is 0. Only -1 and 1 are relatively prime to 0, so if a or b is 0, the method returns `true` only if the other value is -1 or 1.

The code then calls the `GCD` method to get the greatest common divisor of `a` and `b`. If the greatest common divisor is -1 or 1, the values are relatively prime, so the method returns `true`. Otherwise, the method returns `false`.

Now that you know how the method works, implement it and your testing code in your favorite programming language. Did you find any bugs in your initial version of the method or in the testing code? Did you get any benefit from the testing code?

```
def gcd(a, b):
```

```
    a = abs(a)
```

```
    b = abs(b)
```

```
    if a == 0:
```

```
        return b
```

```
    if b == 0:
```

```
        return a
```

```
while True:
    remainder = a % b
    if remainder == 0:
        return b
    a, b = b, remainder

def are_relatively_prime(a, b):
    if a == 0:
        return b == 1 or b == -1
    if b == 0:
        return a == 1 or a == -1
    g = gcd(a, b)
    return g == 1 or g == -1
```

The implementation worked, and testing helped make sure no edge cases were missed.

Problem 8.9, Stephens page 200

Exhaustive testing actually falls into one of the categories black-box, white-box, or gray-box. Which one is it and why?

Black box, as they don't require knowledge of the actual method's code.

Problem 8.11, Stephens page 200

Suppose you have three testers: Alice, Bob, and Carmen. You assign numbers to the bugs so the testers find the sets of bugs {1, 2, 3, 4, 5}, {2, 5, 6, 7}, and {1, 2, 8, 9, 10}. How can you use the Lincoln index to estimate the total number of bugs? How many bugs are still at large?

Use each pair to calculate a Lincoln Index (10, 12.5, and 20) then take the average of ~14 bugs, with ~4 still at large

Problem 8.12, Stephens page 200

What happens to the Lincoln estimate if the two testers don't find any bugs in common? What does it mean? Can you get a lower bound estimate of the number of bugs?

The Lincoln estimate gets a divide by 0 error, and we don't know the number of bugs, but we can assume many since all different ones were found. We can pretend 1 bug was found in common to get a number for the lower bound estimate.